Centro Paula Souza

Fatec
Jacareí
Prof. Francisco de Moura

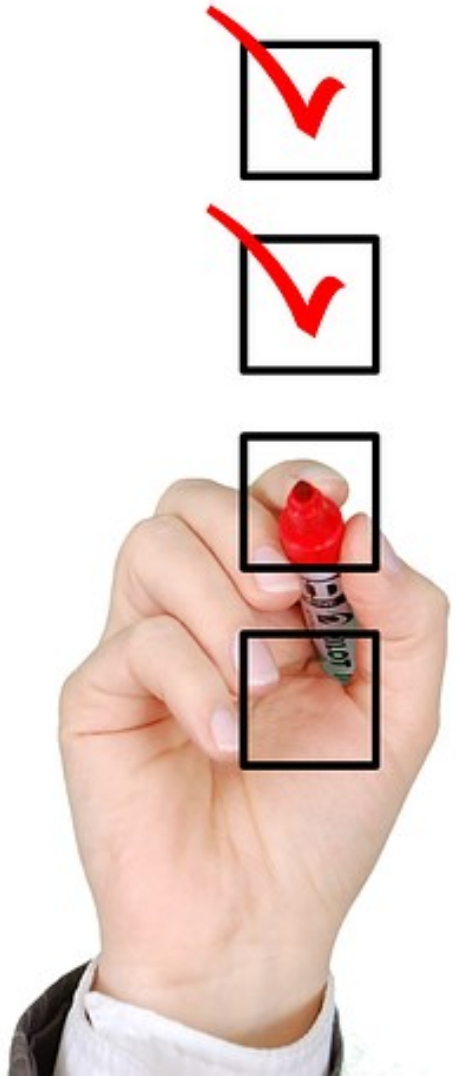
INTEGRAÇÃO E ENTREGA CONTÍNUA

Testes Automatizados no CI/CD

Professora: Lucineide Pimenta

Tópicos da aula

Fatec
Jacareí
Prof. Francisco de Moura



- ✓ Conhecer a importância dos testes automatizados no ciclo de desenvolvimento.
- ✓ Conhecer os principais tipos de testes automatizados (unitário, integração e end-to-end).
- ✓ Demonstrar a implementação de testes automatizados em um pipeline de CI/CD.
- ✓ Conhecer boas práticas para escrita e execução de testes automatizados.
- ✓ Aplicar testes automatizados no projeto ABP.

O que são Testes Automatizados?

Fatec
Jacareí
Prof. Francisco de Moura

- Testes automatizados são scripts que validam o comportamento do software sem necessidade de execução manual.
- Eles garantem que o código continua funcionando corretamente após mudanças, reduzindo erros e melhorando a eficiência do desenvolvimento.
- **Principais benefícios dos testes automatizados:**
  - Redução de erros humanos.
  - Feedback rápido sobre a qualidade do código.
  - Facilidade na manutenção e evolução do software.
  - Maior confiança no código antes da entrega.

🔗 **Referência:** https://martinfowler.com/articles/practical-test-pyramid.html

Por que os Testes Automatizados são Essenciais?

Fatec
Jacareí
Prof. Francisco de Moura

- Sem testes automatizados, cada nova funcionalidade pode quebrar partes do sistema sem que os desenvolvedores percebam.
- Isso resulta em bugs em produção, impactando usuários e aumentando os custos de correção.
- **Exemplo real:** *Uma grande empresa de e-commerce sofreu uma falha crítica ao lançar uma promoção. Um erro no código impediu que cupons de desconto fossem aplicados corretamente. Como não havia testes automatizados cobrindo essa funcionalidade, o erro passou despercebido e gerou prejuízos financeiros.*

🔗 **Referência:** https://testing.googleblog.com/

Por que os Testes Automatizados são Essenciais?

Fatec
Jacareí
Prof. Francisco de Moura

- **Por que os testes automatizados são essenciais?**
  - Reduzem a incidência de bugs em produção.
  - Garantem que novas implementações não quebrem funcionalidades existentes.
  - Aceleram a entrega de software com qualidade.
  - Melhoram a confiabilidade do código.
- **Exemplo real:** *Uma fintech implementou testes automatizados para validar transações bancárias. Antes disso, erros só eram detectados em produção. Com a automação, os problemas passaram a ser identificados e corrigidos ainda na fase de desenvolvimento.*

🔗 **Referência:** Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.

O Papel dos Testes Automatizados no Desenvolvimento Ágil e CI/CD

Fatec
Jacareí
Prof. Francisco de Moura

- No desenvolvimento ágil e DevOps, entregas rápidas são essenciais.
- Testes automatizados permitem validar código continuamente, garantindo qualidade sem comprometer a velocidade do desenvolvimento.
- **Fluxo de trabalho ideal:**
  - O desenvolvedor escreve código e faz um commit.
  - O GitHub Actions executa testes automaticamente.
  - Se os testes falharem, a alteração não é aprovada.
  - Se os testes passarem, o código é mesclado e pode ser implantado.

🔗 **Referência:** https://www.atlassian.com/continuous-delivery/software-testing

INTEGRAÇÃO E ENTREGA CONTÍNUA

Tipos de Testes Automatizados

Tipos de Testes Automatizados

Fatec
Jacareí
Prof. Francisco de Moura

Os testes podem ser categorizados em diferentes níveis, garantindo cobertura total do sistema:

1. Testes Unitários:

- Validam pequenas partes do código (exemplo: funções e métodos).
- Rápidos e fáceis de rodar.
- *Exemplo real: Testar uma função que soma dois números.*

2. Testes de Integração:

- Verificam a interação entre componentes do sistema.
- *Exemplo real: Validar se a API retorna os dados esperados do banco de dados.*

3. Testes End-to-End (E2E):

- Simulam ações de um usuário real.
- *Exemplo real: Automatizar um teste para verificar se um usuário consegue fazer login e concluir uma compra em um site.*

🔗 **Referência:** https://www.selenium.dev/documentation/en/

🔗 **Referência:** https://martinfowler.com/bliki/TestPyramid.html

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

1. Testes Unitários:

- Validam pequenas partes do código (exemplo: funções e métodos).
- Rápidos e fáceis de rodar.
- Exemplo real: Testar uma função que soma dois números.
- Exemplo: Testar se uma função de cálculo retorna o valor correto.

🔗 **Referência:** https://www.selenium.dev/documentation/en/

🔗 **Referência:** https://martinfowler.com/bliki/TestPyramid.html

Tipos de Testes Automatizados

Fatec
Jacareí
Prof. Francisco de Moura

Passo a passo para criar um Teste Unitário em Python com PyTest:

1. Instale o PyTest caso ainda não tenha:

   pip install pytest

2. Crie a função a ser testada (exemplo: função que soma dois números).

```
def somar(a, b):
return a + b
```

Tipos de Testes Automatizados

Fatec
Jacareí
Prof. Francisco de Moura

3. Crie um arquivo de teste **(test_calculos.py)** e escreva o teste unitário:

```
import pytest
from calculos import somar

def test_somar():
    assert somar(2, 3) == 5
    assert somar(-1, 1) == 0
```

Tipos de Testes Automatizados

Fatec
Jacareí
Prof. Francisco de Moura

4. Execute o teste:

pytest test_calculos.py

- **Caso Real:** Empresas como Google e Facebook utilizam testes unitários para validar funções críticas em sistemas de busca e algoritmos de recomendação.

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

2. Testes de Integração:

- Verificam a interação entre componentes do sistema.
- Exemplo real: Validar se a API retorna os dados esperados do banco de dados.
- Avaliam a comunicação entre diferentes partes do sistema.
- Exemplo: Verificar se a API se comunica corretamente com o banco de dados.

🔗 **Referência:** https://www.selenium.dev/documentation/en/

🔗 **Referência:** https://martinfowler.com/bliki/TestPyramid.html

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

Passo a passo para criar um Teste de Integração:

1. Simule a interação entre dois módulos.
   Por exemplo, um sistema de login e um banco de dados.

2. Crie o código para autenticação **(auth.py):**

```
class AuthService:
    def login(self, user, password):
        if user == "admin" and password == "123":
            return "Token123"
        return None
```

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

3. Crie o teste de integração **(test_auth.py)** para verificar a conexão entre módulos:

```
from auth import AuthService

def test_login():
    auth = AuthService()
    assert auth.login("admin", "123") == "Token123"
    assert auth.login("user", "wrong_pass") is None
```

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

4. **Execute o teste:**

   pytest test_auth.py

- **Caso Real:** Testes de integração são essenciais para garantir que APIs de pagamento, como PayPal ou Stripe, funcionem corretamente dentro de um sistema eCommerce.

🔗 **Referência:** Testing Python Applications

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

3. Testes End-to-End (E2E):

- Simulam ações de um usuário real.
- *Exemplo real: Automatizar um teste para verificar se um usuário consegue fazer login e concluir uma compra em um site.*

🔗 **Referência:** https://www.selenium.dev/documentation/en/

🔗 **Referência:** https://martinfowler.com/bliki/TestPyramid.html

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

Passo a passo para criar um Teste de Aceitação com Selenium:

1. Instale as dependências:

   pip install selenium

2. Baixe o WebDriver (exemplo: ChromeDriver) e coloque-o no PATH.

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

3. **Escreva o script de teste** (test_login.py) **para simular um usuário acessando um site:**

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://exemplo.com/login")

username = driver.find_element(By.NAME, "username")
password = driver.find_element(By.NAME, "password")
login_button = driver.find_element(By.NAME, "login")

username.send_keys("admin")
password.send_keys("123")
login_button.click()

assert "Dashboard" in driver.title

driver.quit()
```

Tipos de Testes Automatizados (Visão Geral)

Fatec
Jacareí
Prof. Francisco de Moura

4. Execute o teste:
   python test_login.py

- **Caso Real:** Empresas como Netflix e Amazon utilizam testes de aceitação para validar fluxos de compra, login e navegação em seus sites.

🔗 **Referência:** Selenium Documentation

INTEGRAÇÃO E ENTREGA CONTÍNUA

Ferramentas para Testes Automatizados

Ferramentas para Testes Automatizados

Fatec
Jacareí
Prof. Francisco de Moura

- **O que são?**
  Ferramentas de testes automatizados são softwares que permitem escrever, executar e monitorar testes de forma automática, garantindo qualidade no desenvolvimento de aplicações.
- **Benefícios:**
  - Redução de erros humanos
  - Execução rápida e repetível de testes
  - Identificação precoce de bugs
  - Maior confiabilidade no código

🔗 **Referência:** Guia Completo sobre Testes Automatizados

Ferramentas para Testes Automatizados

Fatec
Jacareí
Prof. Francisco de Moura

- **Testes Unitários:**
  - Jest (JavaScript)
  - JUnit (Java)
  - PyTest (Python)
- **Testes de Integração:**
  - Postman
  - Newman
  - Supertest
- **Testes End-to-End:**
  - Cypress
  - Selenium
  - Playwright

🔗 **Referência:**
https://testing.googleblog.com/

Selenium - Testes de Interface Gráfica (GUI)

Fatec
Jacareí
Prof. Francisco de Moura

- **O que é?**
  - O **Selenium** é uma ferramenta open-source usada para automação de testes de aplicações web, simulando a interação do usuário.
- **Como funciona?**
  - Usa WebDrivers para interagir com navegadores (Chrome, Firefox, Edge)
  - Suporta linguagens como Python, Java e JavaScript

- **Caso real:** Empresas como Netflix usam Selenium para testar a experiência do usuário na plataforma.

🔗 **Referência:** Selenium Documentation

Selenium - Testes de Interface Gráfica (GUI)

Fatec
Jacareí
Prof. Francisco de Moura

- **Exemplo Prático: Automação de Login**

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://exemplo.com/login")

driver.find_element(By.NAME, "username").send_keys("admin")
driver.find_element(By.NAME, "password").send_keys("123")
driver.find_element(By.NAME, "login").click()

assert "Dashboard" in driver.title
driver.quit()
```

JUnit - Testes Unitários em Java

Fatec
Jacareí
Prof. Francisco de Moura

- **O que é?**
  - O **JUnit** é uma das principais ferramentas para testes unitários em Java. Ele permite validar pequenos blocos de código isoladamente.
- **Principais Recursos:**
  - Estrutura de asserções para verificar resultados esperados
  - Configuração e execução simplificada de testes

- **Caso real:** Empresas financeiras utilizam JUnit para validar cálculos em sistemas bancários.

🔗 **Referência:** JUnit 5 User Guide

JUnit - Testes Unitários em Java

Fatec
Jacareí
Prof. Francisco de Moura

- **Exemplo Prático: Testando uma Calculadora**

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class CalculadoraTest {
    @Test
    public void testSoma() {
        Calculadora calc = new Calculadora();
        assertEquals(5, calc.soma(2, 3));
    }
}
```

PyTest - Testes Unitários e de Integração em Python

Fatec
Jacareí
Prof. Francisco de Moura

- **O que é?**
  - O **PyTest** é uma biblioteca para testes em Python, amplamente usada para testar funções e módulos de forma simples e eficaz.
- **Principais Recursos:**
  - Suporte para testes unitários e de integração
  - Estrutura simples e fácil de usar
  - Execução paralela de testes

- **Caso real:** Startups utilizam PyTest para testar APIs e garantir comunicação entre serviços.

🔗 **Referência:** PyTest Documentation

PyTest - Testes Unitários e de Integração em Python

Fatec
Jacareí
Prof. Francisco de Moura

- **Exemplo Prático: Teste de uma API REST**

```
import requests

def test_api_status():
    response = requests.get("https://jsonplaceholder.typicode.com/posts/1")
    assert response.status_code == 200
```

Comparação e Escolha da Ferramenta Certa

Fatec
Jacareí
Prof. Francisco de Moura

- **Comparação entre Ferramentas**

Ferramenta	Tipo de Teste	Linguagem Principal	Caso de Uso
Selenium	Interface gráfica (E2E)	Python, Java, C#	Testes de UI
JUnit	Unitário	Java	Testes de métodos/classes
PyTest	Unitário e Integração	Python	Testes de APIs e lógica de negócio

- **Qual escolher?**
  - Se deseja testar **interfaces gráficas**, use **Selenium**.
  - Se trabalha com **Java**, use **JUnit** para testes unitários.
  - Se desenvolve em **Python**, use **PyTest** para testes rápidos e eficazes.
- **Dica:** O ideal é combinar ferramentas para uma estratégia de testes completa!

🔗 **Referência:** Comparação de Ferramentas de Testes

INTEGRAÇÃO E ENTREGA CONTÍNUA

Implementando Testes Automatizados no CI/CD

Implementando Testes Automatizados no CI/CD

Fatec
Jacareí
Prof. Francisco de Moura

- **1. Criar um Repositório no GitHub**
- **2. Criar o Código de Testes**
- **3. Configurar um Workflow no GitHub Actions**

name: Testes Automatizados
on: [push]
jobs:
test:
runs-on: ubuntu-latest
steps:
- name: Checkout do código
uses: actions/checkout@v2
-name: Instalar dependências
run: npm install
- name: Executar testes
run: npm test

```yaml
name: Testes Automatizados
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do código
        uses: actions/checkout@v2
      - name: Instalar dependências
        run: npm install
      - name: Executar testes
        run: npm test
```

- **4. Validar a Execução dos Testes**
  - **Acesse Action no GitHub e verifique os logs de execução.**

🔗 **Referência:** https://docs.github.com/en/actions/guides

Introdução à Integração de Testes no CI/CD

Fatec
Jacareí
Prof. Francisco de Moura

- **O que é CI/CD?**
  - CI/CD (Continuous Integration/Continuous Deployment) é uma prática DevOps que automatiza a construção, teste e entrega de software.
- **Por que integrar testes no CI/CD?**
  - Detecta falhas rapidamente
  - Reduz riscos na entrega de software
  - Automatiza a validação do código
  - Garante qualidade contínua

🔗 **Referência:** O que é CI/CD?

Configurando um Repositório no GitHub

Fatec
Jacareí
Prof. Francisco de Moura

- **Passo 1: Criando o Repositório**
  1. Acesse GitHub e clique em **New Repository**.
  2. Defina um nome e escolha "Public" ou "Private".
  3. Inicialize com um README e selecione **.gitignore** para sua linguagem.
- **Passo 2: Clonando o Repositório**

  git clone https://github.com/seu-usuario/seu-repo.git

  cd seu-repo

- **Caso real:** Equipes ágeis usam GitHub para versionar código e integrar testes.

🔗 **Referência:** Criando Repositórios no GitHub

Configurando um Workflow do GitHub Actions

Fatec
Jacareí
Prof. Francisco de Moura

- **Passo 3: Criando o Arquivo do Workflow**
  1. No repositório, vá para **Actions** → **New Workflow**.
  2. Clique em "Set up a workflow yourself".
  3. Crie um arquivo **.github/workflows/ci.yml** com o seguinte conteúdo ao lado:

```yaml
name: CI Pipeline

on:
  push:
    branches:
      - main
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Configurar ambiente
        run: |
          python -m venv venv
          source venv/bin/activate
          pip install -r requirements.txt
      - name: Executar Testes
        run: pytest tests/
```

- **Caso real:** Empresas usam GitHub Actions para automação de testes antes do deploy.

🔗 **Referência:** GitHub Actions

Criando Testes Unitários com PyTest

Fatec
Jacareí
Prof. Francisco de Moura

- **Passo 4: Criando Testes Unitários**

1. Instale o PyTest:

```
pip install pytest
```

2. Crie um arquivo **tests/test_app.py**:

```
def soma(a, b):
    return a + b

def test_soma():
    assert soma(2, 3) == 5
```

3. Execute os testes:

```
pytest
```

- ***Caso real:*** Equipes de desenvolvimento utilizam PyTest para validar funções críticas.

🔗 **Referência:** PyTest Documentation

Criando Testes de Integração com PyTest e Requests

Fatec
Jacareí
Prof. Francisco de Moura

- **Passo 5: Testando uma API**
  1. Instale a biblioteca Requests:

     pip install requests
  2. Crie **tests/test_api.py**:

```
import requests

def test_api():
    response = requests.get("https://jsonplaceholder.typicode.com/posts/1")
    assert response.status_code == 200
```

3. ***Execute os testes:***

   pytest tests/

- **Caso real:** Empresas usam testes de integração para garantir a comunicação entre serviços.

🔗 **Referência:** Requests Documentation

Criando Testes End-to-End com Selenium

Fatec
Jacareí
Prof. Francisco de Moura

- **Passo 6: Automatizando Testes no Navegador**
  1. Instale o Selenium:
     pip install selenium
  2. Baixe o WebDriver do Chrome aqui.
  3. Crie **tests/test_ui.py**:

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://exemplo.com")
assert "Exemplo" in driver.title
driver.quit()
```

- **Caso real:** Selenium é usado para testar formulários de login e checkout.

🔗 **Referência:** Selenium Documentation

Adicionando o Relatório de Testes

Fatec
Jacareí
Prof. Francisco de Moura

- **Passo 7: Gerando Relatórios**
  1. Instale pytest-html:

     pip install pytest-html
  2. Execute os testes gerando relatório:

     pytest --html=report.html
  3. Abra report.html no navegador para visualizar os resultados.

- **Caso real:** Equipes geram relatórios automáticos para auditorias e qualidade de código.

🔗 **Referência:** pytest-html

Configurando o CI/CD para Executar os Testes

Fatec
Jacareí
Prof. Francisco de Moura

- **Passo 8: Atualizando o Workflow do GitHub Actions**
  Modifique **.github/workflows/ci.yml** para salvar o relatório:

```
- name: Executar Testes
    run: pytest --html=report.html

  - name: Salvar Relatório
    uses: actions/upload-artifact@v3
    with:
      name: relatorio-teste
      path: report.html
```

- **Caso real:** Organizações salvam relatórios de testes para auditorias de segurança.

🔗 **Referência:** Actions Artifacts

Configurando o Deploy Automático Após Testes

Fatec
Jacareí
Prof. Francisco de Moura

- **Passo 9: Adicionando Deploy ao CI/CD**
  - Modifique ci.yml para incluir um job de deploy:

```
deploy:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v3
    - name: Deploy no Servidor
      run: |
        echo "Deploy realizado com sucesso!"
```

- **Caso real:** Empresas integram testes e deploy para evitar falhas em produção.

🔗 **Referência:** Continuous Deployment

Conclusão - O que Aprendemos?

Fatec
Jacareí
Prof. Francisco de Moura

- **Resumo do Processo:**
  - Criamos um repositório GitHub
  - Configuramos GitHub Actions para CI/CD
  - Criamos testes unitários, de integração e E2E
  - Adicionamos relatórios
  - Implementamos um pipeline completo
- **Próximos Passos:**
  - Experimentar outras ferramentas (Jenkins, CircleCI)
  - Melhorar cobertura de testes
  - Implementar testes de carga e segurança

🔗 **Referências:**

- CI/CD Pipeline
- GitHub Actions

INTEGRAÇÃO E ENTREGA CONTÍNUA

Boas Práticas para Testes Automatizados

Boas Práticas para Testes Automatizados

Fatec
Jacareí
Prof. Francisco de Moura

1. Escrever testes claros e independentes.
2. Utilizar mocks e stubs para isolar dependências.
3. Garantir que os testes rodem rapidamente.
4. Monitorar a cobertura de testes.
5. Evitar falsos positivos e negativos nos testes.

🔗 **Referência:** https://kentcdodds.com/blog/write-tests

INTEGRAÇÃO E ENTREGA CONTÍNUA

Atividade Prática (Individual)

Bibliografia Básica

Fatec
Jacareí
Prof. Francisco de Moura

- HUMBLE J; PRIKLANDNICKI R. **Entrega Contínua:** Como Entregar Software de Forma Rápida e Confiável. São Paulo: Bookman, 2013.
- MUNIZ, A.; et al. **Jornada DevOps:** Unindo Cultura Ágil, Lean e Tecnologia Para Entrega de Software Com Qualidade. São Paulo: Brasport, 2019.
- SATO D. **DevOps na prática:** entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.
- SILVA, R. **Entrega contínua em Android:** Como automatizar a distribuição de apps. São Paulo: Casa do Código, 2016.

Bibliografia Complementar

Fatec
Jacareí
Prof. Francisco de Moura

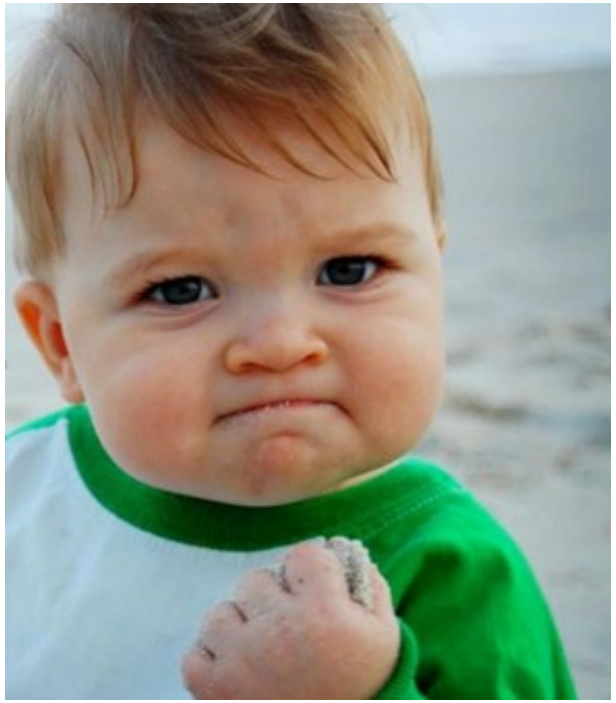
- ARUNDEL, J. DOMINGUS, J. **DevOps nativo de nuvem com Kubernetes**. São Paulo: Novatec, 2019.
- MORAES, G. **Caixa de Ferramentas DevOps:** Um guia para construção, administração e arquitetura de sistemas modernos. São Paulo: Casa do Código, 2015. PIRES, A.; MILITÃO, J. **Integração Contínua com Jenkins**. São Paulo: Casa do Código, 2019.
- VITALINO, J. F. N.; CASTRO, M. A. N. **Descomplicando o Docker.** 2 ed. São Paulo: Brasport, 2018.
- SILVERMAN, R. E. **Git:** guia prático**.** São Paulo: Novatec, 2019.

Dúvidas?

Fatec
Jacareí
Prof. Francisco de Moura

Considerações Finais

Fatec
Jacareí
Prof. Francisco de Moura



Professora:

Lucineide Pimenta

Bom descanso à todos!